Instituto Politécnico Nacional
CECyT 9 “Juan de Dios Bátiz”

PROGRAMACIÓN WEB · ENTREGA DEL MÓDULO

NOM-COLAB: módulo web conectado a la base de datos, publicado en Azure y protegido con roles

Gestión de cuentas: registro, inicio de sesión y administración de usuarios



Equipo 6 “WACK” · 5IM7
We carry the flame

Integrantes
González Azuara Arturo Leonardo
Martínez Ortega Uriel Alejandro
Rodríguez Esparza Diego Alejandro
Soria Flores Josep Aarón

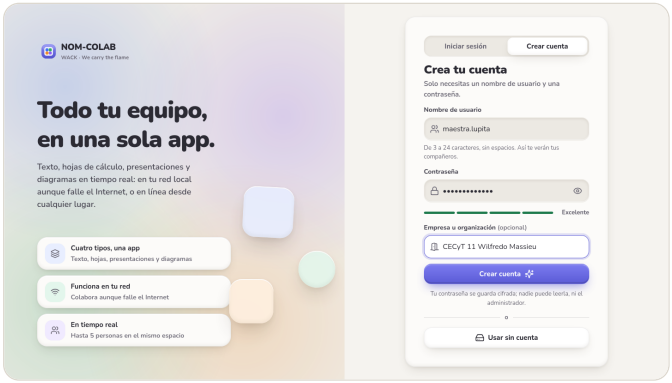
Profesor: Juan Ángel Calvillo Pérez
Página: https://nomcolab-wack.azurewebsites.net
Fecha: 22 de octubre de 2026

1. Lo que se pidió y lo que se entrega

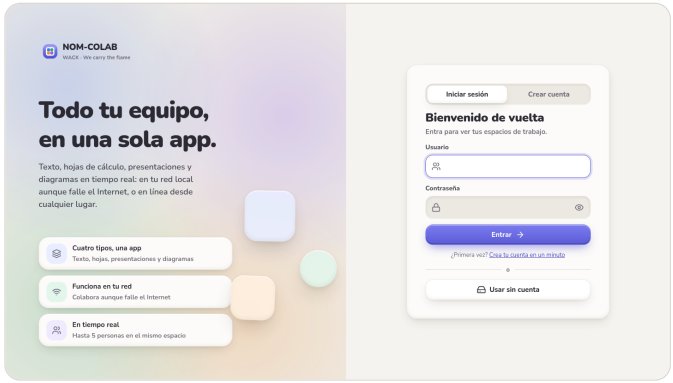
Requisito	Cómo se cumple
Un módulo de la página web	**Gestión de cuentas** de NOM-COLAB: registro, inicio de sesión, cierre de sesión, preferencias y la pantalla **Administrar usuarios**.
Conectado a la base de datos	Usuarios, roles, sesiones y preferencias viven en **Azure SQL Database** (4 tablas). Cada consulta va a la base; nada de esto se guarda en archivos.
Subido a Azure	**Azure App Service** (Linux, Java 21, plan F1 gratuito) en `nomcolab-wack.azurewebsites.net`, solo por HTTPS.
Seguridad aplicada con roles	Tabla `rol` (ADMINISTRADOR y USUARIO). La base de datos "jala" el rol en cada petición; la API de administración responde **403** a quien no sea administrador y la interfaz ni siquiera le muestra la opción.

2. El módulo: Gestión de cuentas

NOM-COLAB es una suite colaborativa (texto, hojas de cálculo, presentaciones y diagramas) para docentes del CECyT 11. El módulo de cuentas es la puerta de entrada: sin una sesión válida no se puede ver ningún espacio ni documento.



Crear cuenta: nombre, contraseña (con medidor de fuerza) y empresa opcional.



Iniciar sesión. El mensaje de error es genérico para no revelar qué usuarios existen.

Casos de uso del módulo

CU	Caso	Qué hace
CU-01	Crear usuario	Valida nombre (3–24 caracteres) y contraseña (mín. 8), revisa que el nombre esté libre, guarda el hash y asigna el rol USUARIO.
CU-02	Iniciar sesión	Busca la cuenta con su rol (JOIN), compara el hash y crea una sesión con token y fecha de expiración.
CU-03	Cerrar sesión	Borra la sesión de la base de datos; el token deja de servir.

CU-04	Bloquear acceso	Al 6.º intento fallido bloquea la cuenta 15 minutos.
CU-05	Cifrar credenciales	PBKDF2-HMAC-SHA256, 120 000 iteraciones, sal aleatoria de 16 bytes por cuenta.
CU-30	Administrar usuarios y roles	Solo ADMINISTRADOR: lista las cuentas, cambia el rol y desbloquea cuentas.

3. Arquitectura

Navegador · celular · app Android
HTML, CSS y JavaScript. Solo guarda el token de sesión.

→

Azure App Service
Java 21 + Spring Boot 4. API REST `/api/cuentas` y `/api/admin`. Filtros de sesión y de rol.

→

Azure SQL Database
Tablas `rol`, `usuario`, `sesion`, `preferencias`. Conexión JDBC cifrada.

El mismo programa funciona también en una red local sin Internet: ahí usa una base de datos **H2** embebida con las mismas tablas. En Azure se cambia a SQL Server solo con variables de configuración; el código es el mismo.

4. Diseño de la base de datos

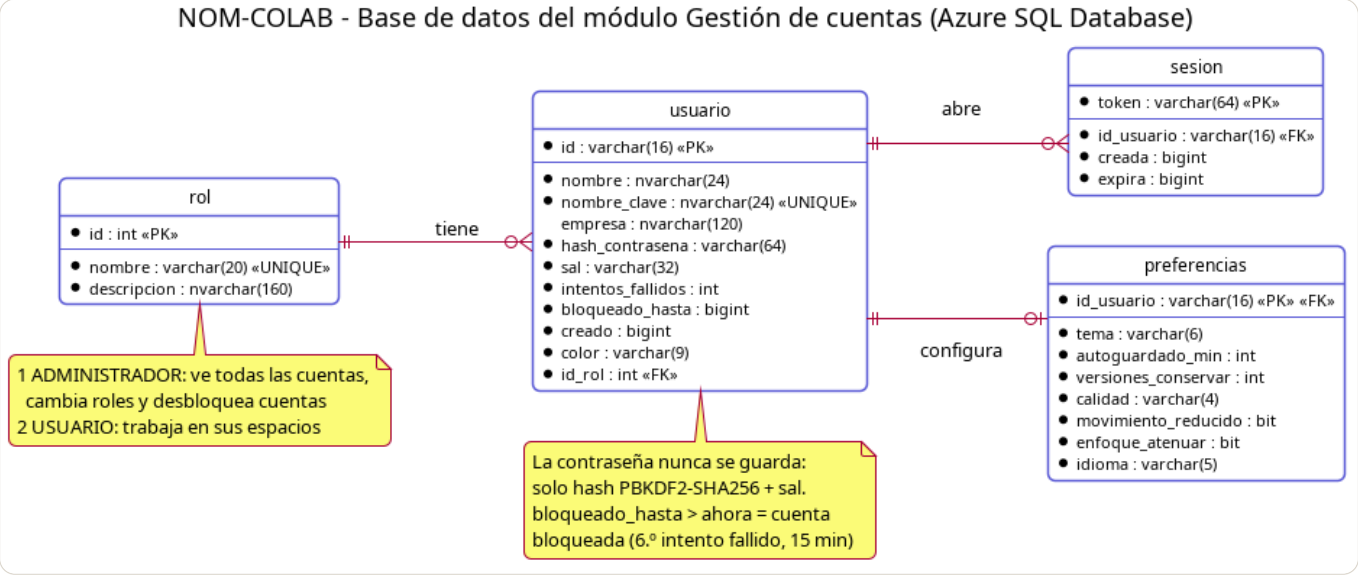


Diagrama entidad-relación del módulo de cuentas en Azure SQL.

Tabla	Para qué sirve	Llaves
`rol`	Catálogo de roles: 1 ADMINISTRADOR, 2 USUARIO, con su descripción.	PK `id`; `nombre` único
`usuario`	Cuenta: nombre, empresa, hash y sal de la contraseña, intentos fallidos, bloqueo y rol.	PK `id`; FK `id_rol → rol.id`; `nombre_clave` único
`sesion`	Sesiones abiertas: token aleatorio de 256 bits, inicio y expiración (72 h).	PK `token`; FK `id_usuario` (borrado en cascada)

`preferencias`	Tema, autoguardado, versiones a conservar, calidad, movimiento.	PK/FK `id_usuario`

Cómo se “jala” el rol (consulta real del repositorio)

```
SELECT u.id, u.nombre, u.hash_contrasena, u.sal, u.intentos_fallidos, u.bloqueado_hasta,
       r.nombre AS rol, p.tema, p.autoguardado_min, ...
  FROM usuario u
  JOIN rol r ON r.id = u.id_rol
  LEFT JOIN preferencias p ON p.id_usuario = u.id
 WHERE u.nombre_clave = ?          -- parámetro: nunca se concatena texto del usuario
```

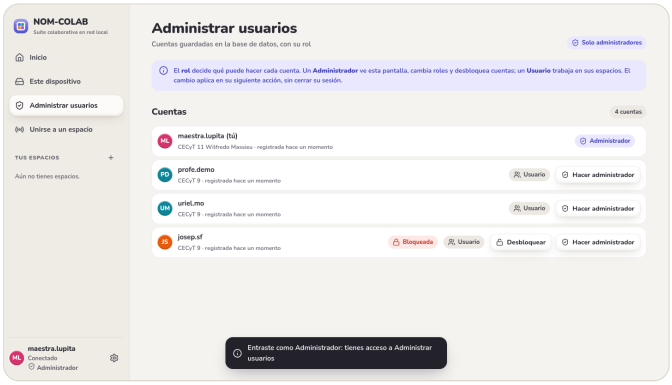
5. Seguridad por roles

Acción	Visitante	USUARIO	ADMINISTRADOR
Registrarse e iniciar sesión	**Sí**	**Sí**	**Sí**
Espacios y documentos propios o compartidos	No (401)	**Sí**	**Sí**
Ver todas las cuentas	No (401)	No (403)	**Sí**
Cambiar roles y desbloquear cuentas	No (401)	No (403)	**Sí (no puede quitarse su propio rol)**

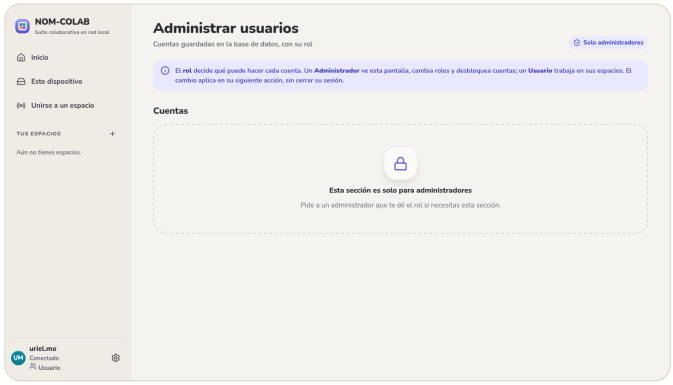
Cómo funciona, paso a paso

1. **FiltroSesion** revisa el token de cada petición a `/api/**` (salvo registro e inicio de sesión). Sin token o con uno vencido o inventado: **401**.
2. Con el token busca la sesión en la base y carga al usuario **con su rol** (el JOIN de arriba).
3. **FiltroRol** protege `/api/admin/**`: si el rol no es ADMINISTRADOR responde **403 “Esta sección es solo para administradores”**.
4. Como el rol se relee en cada petición, un cambio de rol aplica de inmediato, sin cerrar la sesión.
5. La interfaz no muestra “Administrar usuarios” a un USUARIO; aunque escriba la dirección a mano, el servidor la niega.

```
@Component
public class FiltroRol implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        Usuario u = (Usuario) req.getAttribute(FiltroSesion.ATRIBUTO);   // lo dejó FiltroSesion
        if (u == null || !u.esAdministrador())
            throw ErrorNom.prohibido("Esta sección es solo para administradores");   // → 403
        return true;
    }
}
```



ADMINISTRADOR: cuentas de la BD con su rol; “Hacer administrador” y “Desbloquear”.



USUARIO que escribe la dirección a mano: el servidor responde 403.

6. Otras medidas de seguridad de la base de datos y la aplicación

Medida	Detalle
Contraseñas	Nunca se guardan: solo el hash PBKDF2-HMAC-SHA256 (120 000 iteraciones) con sal propia de cada cuenta. Comparación en tiempo constante.
Inyección SQL	Todas las consultas usan parámetros (`?`) con `JdbcClient` ; el texto del usuario nunca se concatena al SQL.
Validación de datos	Nombre de 3–24 letras, números, punto o guion (rechaza HTML/scripts); contraseña de 8 caracteres mínimo.
Fuerza bruta	Bloqueo de 15 minutos al 6.º intento; el mensaje de error no revela si el usuario existe.
Sesiones	Token aleatorio de 256 bits, guardado en la BD, con expiración de 72 h; al cerrar sesión se borra.
Secretos fuera del código	URL, usuario y contraseña de la BD y la cuenta de administrador inicial son *App settings* de Azure (variables de entorno).
Transporte cifrado	La página solo acepta HTTPS (HTTP redirige con 301) y la conexión a la BD usa `encrypt=true` .
Firewall de Azure SQL	Solo acepta conexiones de servicios de Azure (la Web App) y de la IP del equipo de desarrollo.
Datos locales por cuenta	Las copias que el navegador guarda para trabajar sin red se separan por cuenta: otra persona en el mismo equipo no las ve.

7. Despliegue en Azure

Se siguió el mismo camino de la práctica 4 (Git y Azure con la CLI `az`), pero ahora con un servidor Java y una base de datos. La suscripción *Azure for Students* del IPN solo permite ciertas regiones; se usó **westus**.

Recurso	Nombre	Plan y costo

Grupo de recursos	`rg-nomcolab`	—
Servidor y BD	`nomcolab-sql-6a073f` / `nomcolab`	Serverless, oferta gratuita: $0. Si se agota el mes, se pausa (no cobra).
Plan de App Service	`plan-nomcolab`	Linux F1 Free: $0
Web App	`nomcolab-wack`	Java 21, solo HTTPS

```
az login --use-device-code                          # cuenta @alumno.ipn.mx con MFA
az provider register -n Microsoft.Sql --wait
az group create -n rg-nomcolab -l westus
az sql server create -g rg-nomcolab -n nomcolab-sql-6a073f -l westus -u nomadmin -p ********
az sql db create -g rg-nomcolab -s nomcolab-sql-6a073f -n nomcolab --compute-model Serverless \
   -e GeneralPurpose -f Gen5 -c 1 --use-free-limit --free-limit-exhaustion-behavior AutoPause
az sql server firewall-rule create ... --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
az appservice plan create -g rg-nomcolab -n plan-nomcolab --is-linux --sku F1
az webapp create -g rg-nomcolab -p plan-nomcolab -n nomcolab-wack --runtime "JAVA:21-java21"
az webapp config appsettings set ... SPRING_DATASOURCE_URL=... SPRING_DATASOURCE_PASSWORD=***
NOM_DB_PLATAFORMA=sqlserver
az webapp deploy -g rg-nomcolab -n nomcolab-wack --src-path nom-colab.jar --type jar
```

Al arrancar, la aplicación crea las tablas si no existen (`schema-sqlserver.sql`, idempotente) y la cuenta de administrador inicial desde las variables de configuración.

8. Pruebas de seguridad y funcionamiento

Se automatizaron con un navegador real (Selenium) y peticiones directas a la API: **35 de 35 aprobadas** en la prueba integral. Las de roles, sesión y HTTPS se verificaron también contra Azure.

Prueba	Esperado	Resultado
Pedir datos sin token	401	**Aprobada**
Token inventado	401	**Aprobada**
USUARIO entra a `/api/admin/usuarios`	403	**Aprobada**
ADMINISTRADOR entra a `/api/admin/usuarios`	200	**Aprobada**
Inyección SQL en el login (`' OR 1=1 --`)	401	**Aprobada**
Registro con nombre que contiene HTML	400	**Aprobada**
Contraseña de menos de 8 caracteres	400	**Aprobada**
6 intentos fallidos → bloqueo; el administrador desbloquea	423 → 200	**Aprobada**
El administrador intenta quitarse su propio rol	403	**Aprobada**
Cambio de rol aplica con la misma sesión	Acceso inmediato	**Aprobada**
Cuenta sin acceso pide un espacio ajeno	403	**Aprobada**
Permiso de solo lectura intenta editar	403	**Aprobada**

Otra cuenta en el mismo navegador no ve las copias locales de la anterior	No las ve	**Aprobada**
HTTP redirige a HTTPS	301	**Aprobada**
Primer arranque contra Azure SQL: tablas y administrador creados	Tablas creadas	**Aprobada**

9. Cómo probarlo

1. Abrir **https://nomcolab-wack.azurewebsites.net** unos minutos antes (el plan gratuito se duerme; la primera entrada tarda hasta un minuto y el botón dice “Conectando con la base de datos…”).
2. **Crear cuenta** nueva: queda en la base con rol USUARIO. Al entrar no aparece “Administrar usuarios”; si se escribe `…/#/admin`, el servidor responde 403.
3. Entrar con la cuenta de **administrador** (se entrega en persona; no va en este documento): en “Administrar usuarios” aparece la cuenta recién creada, leída de Azure SQL, y se le puede cambiar el rol.

10. Conclusión

El módulo de Gestión de cuentas de NOM-COLAB está publicado en Azure, conectado a una base de datos Azure SQL y protegido con roles que se leen de la base en cada petición: un usuario normal no puede ver ni usar la administración aunque conozca la dirección. Todo corre en servicios gratuitos de la suscripción de estudiante, sin costo.